

Module 6 — Grad Café Analytics on Docker Compose

Vix Talbot (JHED: vtalbot1) · EN.605.256.82 Modern Software Concepts in Python

Repository: github.com/Vix-T/jhu_software_concepts/tree/main/module_6

Docker Hub: hub.docker.com/r/vixbot/module_6 (tags `web-v1`, `worker-v1`)

1. Overview

Module 6 turns the Grad Café analytics app from Modules 3–5 into four cooperating containers. The web service no longer scrapes, loads or computes anything: its two buttons publish tasks to RabbitMQ and return at once, and a separate worker does all writing to PostgreSQL. The whole stack starts with one command on a clean machine, the two application images are published on Docker Hub, and GitHub Actions runs Pylint and Pytest for the module on Python 3.11. The development environment was Docker Desktop 4.47 (Engine 28.4.0, Compose v2.39.4) on an Intel Mac.

2. Docker Compose architecture

Service	Image	Role
db	postgres:16	Holds <code>applicants</code> , <code>ingestion_watermarks</code> and <code>analysis_summary</code> in the named volume <code>pgdata</code> . Health check <code>pg_isready</code> . Its port is not published to the host.
rabbitmq	rabbitmq:3.13-management	Task queue. Health check <code>rabbitmq-diagnostics -q check_port_connectivity</code> , which passes only once the AMQP listener accepts connections. Management UI published on port 15672.
web	vixbot/module_6:web-v1 (build <code>src/web</code>)	Flask app on 0.0.0.0:8080 (published). Reads the stored analysis and applicant rows; the buttons publish tasks. Connects as the read-only role <code>gradcafe_web</code> .
worker	vixbot/module_6:worker-v1 (build <code>src/worker</code>)	Initialises the database at startup, then consumes <code>tasks_q</code> . Mounts <code>src/data</code> (seed JSON) and <code>src/db</code> (shared database modules) read-only. <code>restart: unless-stopped</code> .

`web` and `worker` start only after `db` and `rabbitmq` report healthy (`depends_on: condition: service_healthy`). Each image is built from its own folder, exactly as the assignment's Dockerfiles give it (`python:3.11-slim`, requirements installed first, `USER 1000`). The worker image also bakes in chromedriver for linux64 at build time (build argument `CHROMEDRIVER_VERSION`, default `154.0.8037.92`) with only the four libraries it links against, so nothing is downloaded at runtime. Every setting in `docker-compose.yml` has a development default (`${VAR:-default}`), so the stack starts with no `.env`; the defaults are development-only credentials for a database that is not reachable from outside the compose network, and a `.env` file overrides them.

3. RabbitMQ message flow

1. The browser POSTs `/pull-data` or `/update-analysis`.
2. `publisher.publish_task()` connects with `RABBITMQ_URL`, declares the durable direct exchange `tasks`, the durable queue `tasks_q` and their binding (routing key `tasks`), turns on publisher confirms, and

publishes compact JSON `{"kind", "ts", "payload"}` with `delivery_mode=2` and `mandatory=True`. The connection is closed in `finally`.

3. The route answers **202** `{"status": "queued", "task": ...}`, or **503** if the publish fails (`pika.exceptions.AMQPError`, including an unroutable or nacked message) or `RABBITMQ_URL` is missing.
4. The page shows a "Request queued" banner; its script then polls `GET /api/status` every 2 seconds for up to 60 seconds and reloads the page when either timestamp ("Analysis last updated", "Data last updated") changes.

Task kind	Queued by	Worker handler
<code>scrape_new_data</code>	Pull Data	<code>handle_scrape_new_data</code> : reads the watermark with <code>SELECT ... FOR UPDATE (or payload["since"])</code> , scrapes entries newer than it through the host's Chrome session, inserts them with <code>ON CONFLICT (url) DO NOTHING</code> , advances the watermark with <code>GREATEST</code> , and recomputes the analysis — all in one transaction.
<code>recompute_analytics</code>	Update Analysis	<code>handle_recompute_analytics</code> : runs every analysis query and replaces the single <code>analysis_summary</code> row.

4. Database initialisation

At every start the worker runs `load_data.initialize_database()` under a PostgreSQL advisory lock, so two workers starting together cannot both seed. It creates the three tables if they are missing and, only when `applicants` is empty, loads `/data/applicant_data.json` (60,025 records; 60,024 rows load, one record has no URL) and sets the watermark to the highest Grad Café result ID in the seed, 1,020,478. It then computes the analysis if none is stored, and creates or updates the read-only role `gradcafe_web`: `CONNECT`, `schema USAGE` and `SELECT` on the three tables only. Every run revokes the role's other privileges first, so re-running is idempotent. Until all of this has committed, the web page answers 503 "The database is being initialised... This page refreshes automatically." and reloads every 5 seconds.

5. Worker behaviour

- **Prefetch 1:** `basic_qos(prefetch_count=1)` — one unacknowledged message at a time.
- **One transaction per message:** the handler runs on the worker's connection; nothing is committed inside it.
- **Ack after commit:** the worker commits, then calls `basic_ack`. A test checks from a second database connection, at the moment of the ack, that the data is already committed.
- **Nack without requeue:** a `try/finally` tracks whether the message was acked; if not, it rolls back and calls `basic_nack(requeue=False)`. Expected failures (database errors, the browser failing or not being attached, retries exhausted, a bad payload) are logged and the worker continues; malformed JSON, non-object bodies and unknown kinds are rejected the same way. Anything unexpected propagates after that cleanup and stops the worker, which Docker restarts.
- **Heartbeat 600 s** on the worker's RabbitMQ connection, so a long scrape does not drop it.

6. Verification

Check	Result
Docker installed	<code>docker run hello-world</code> succeeds (Figure 1).
<code>docker compose config</code>	Valid; every variable resolves to its default with no <code>.env</code> .
Clean first start (<code>down -v</code> , then <code>up --build -d</code>)	<code>up</code> finished in 20 s; the page answered 503 while the worker seeded, then 200 about 48 s later. Worker log: 60,024 rows inserted, watermark 1,020,478, initial analysis computed (Q1 = 32,344). Worker and web restarts: 0.
All services up	<code>db</code> and <code>rabbitmq</code> healthy, <code>web</code> on 8080, <code>worker</code> running (Figure 2).
Page	<code>GET /</code> returns 200 with the analysis (Figure 3).
Update Analysis	202 <code>{"status":"queued","task":"recompute_analytics"}</code> ; worker logged the task done; <code>computed_at</code> moved from 18:01:28 to 18:12:48. In a browser the page showed the queued banner (Figure 4), then reloaded by itself and showed the new "Analysis last updated" time.
Pull Data with no Chrome	202; worker logged <code>PullPreconditionError</code> , nacked without requeue and kept running (0 restarts); <code>tasks_q</code> empty, 0 redelivered.
Pull Data with the host Chrome session	From inside the worker, <code>host.docker.internal</code> resolved to 192.168.65.254 and <code>/json/version</code> returned Chrome/154. The pull read 3 pages and inserted 29 new rows; the watermark moved from 1,020,478 to 1,020,507 and the analysis was recomputed in the same transaction.
RabbitMQ topology	Exchange <code>tasks</code> (direct) and queue <code>tasks_q</code> both durable, bound with key <code>tasks</code> , one consumer (Figures 5–8).
Non-root containers	<code>id</code> shows uid 1000 in both <code>web</code> and <code>worker</code> .
Persistence	<code>down</code> then <code>up -d</code> kept the volume: row count, watermark and analysis unchanged; worker logged "applicants already has rows: seed skipped".
Read-only web role	Connected as <code>gradcafe_web</code> , SELECT works on all three tables; INSERT, UPDATE, DELETE and CREATE TABLE are refused by PostgreSQL with permission errors.
Docker Hub	Repository public; <code>web-v1</code> and <code>worker-v1</code> pulled anonymously; the stack ran from the tagged images (page 200, both buttons 202) (Figures 9–10).
Tests and lint	375 tests pass with 100% statement coverage; Pylint 10.00/10. GitHub Actions "Module 6 CI" is green on Python 3.11 (Figure 11).

Pulling and running the published images

```
docker pull vixbot/module_6:web-v1
docker pull vixbot/module_6:worker-v1

git clone https://github.com/Vix-T/jhu_software_concepts.git
cd jhu_software_concepts/module_6
docker compose pull web worker
docker compose up -d --no-build      # or: docker compose up --build
```

Then open <http://localhost:8080> (RabbitMQ management: <http://localhost:15672>, guest/guest, development only). Pull Data additionally needs a Chrome started on the host with `--remote-debugging-port=9222` and the Grad Café Cloudflare check cleared by hand; the README gives the exact command.

7. Known limitations

- A pull collects at most 300 new entries, newest first; if more arrived since the last pull, the older ones are skipped and the advanced watermark means they are not fetched later.
- Scraped rows have no LLM-standardised program/university fields, so questions that use those fields count seed rows only.
- When the host sleeps, the RabbitMQ heartbeat lapses and the worker restarts on wake (observed twice overnight); no message is lost.
- A wrong `WEB_DB_PASSWORD` looks the same to the web as a role not created yet, so the page keeps showing the initialising message.
- The web service runs Flask's development server.

8. Screenshots

```
(base) vixedwards@Mac ~ % docker --version
docker compose version
docker run hello-world
Docker version 28.4.0, build d8eb465
Docker Compose version v2.39.4-desktop.1
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

(base) vixedwards@Mac ~ %
```

Figure 1. Docker 28.4.0 and Compose v2.39.4 installed; `docker run hello-world` succeeds.

```
(base) vixedwards@Mac module_6 % docker compose ps --format "table {{.Service}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
SERVICE      IMAGE                                STATUS              PORTS
db             postgres:16                         Up 56 minutes (healthy)  5432/tcp
rabbitmq       rabbitmq:3.13-management            Up 56 minutes (healthy)  0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
web            module_6-web                        Up 56 minutes        0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
worker        module_6-worker                    Up 52 minutes
(base) vixedwards@Mac module_6 %
```

Figure 2. `docker compose ps`: db and rabbitmq healthy, web publishing 8080, worker running (taken before the images were tagged for Docker Hub, hence the local image names).

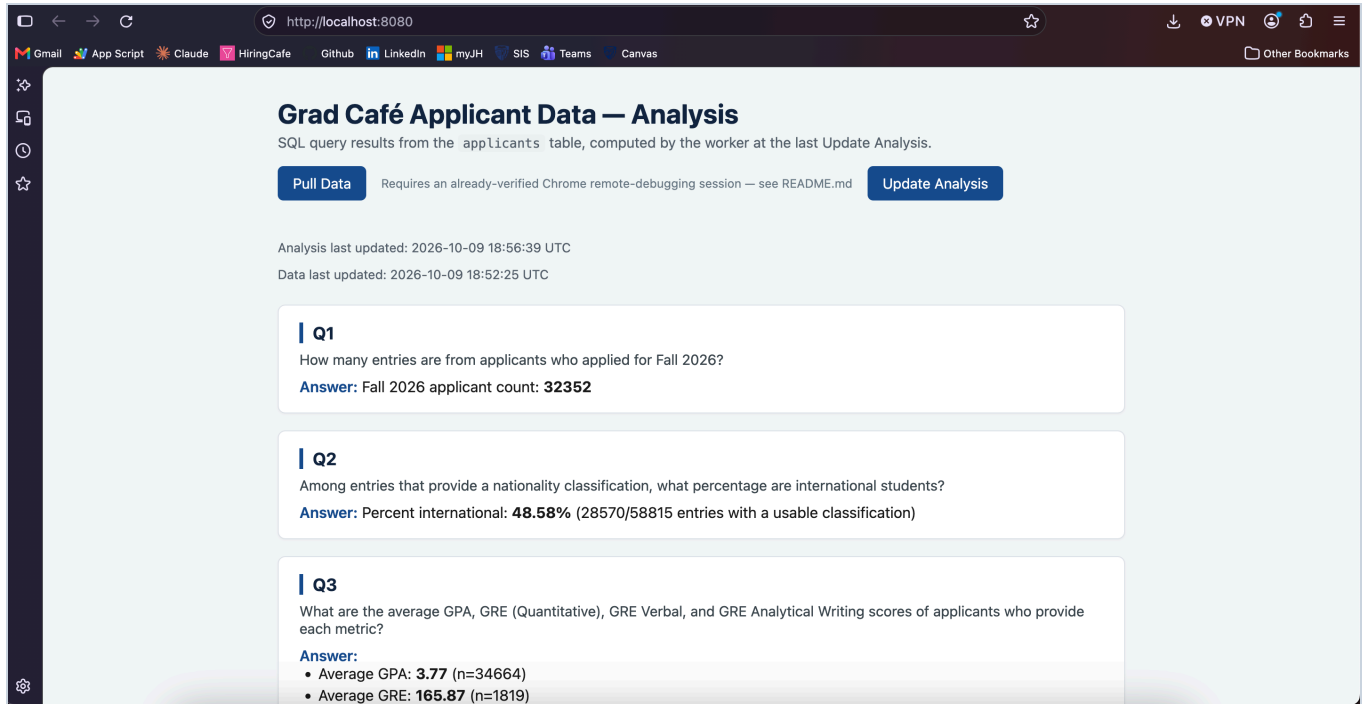


Figure 3. The analysis page at <http://localhost:8080>, rendered from the worker's stored analysis.

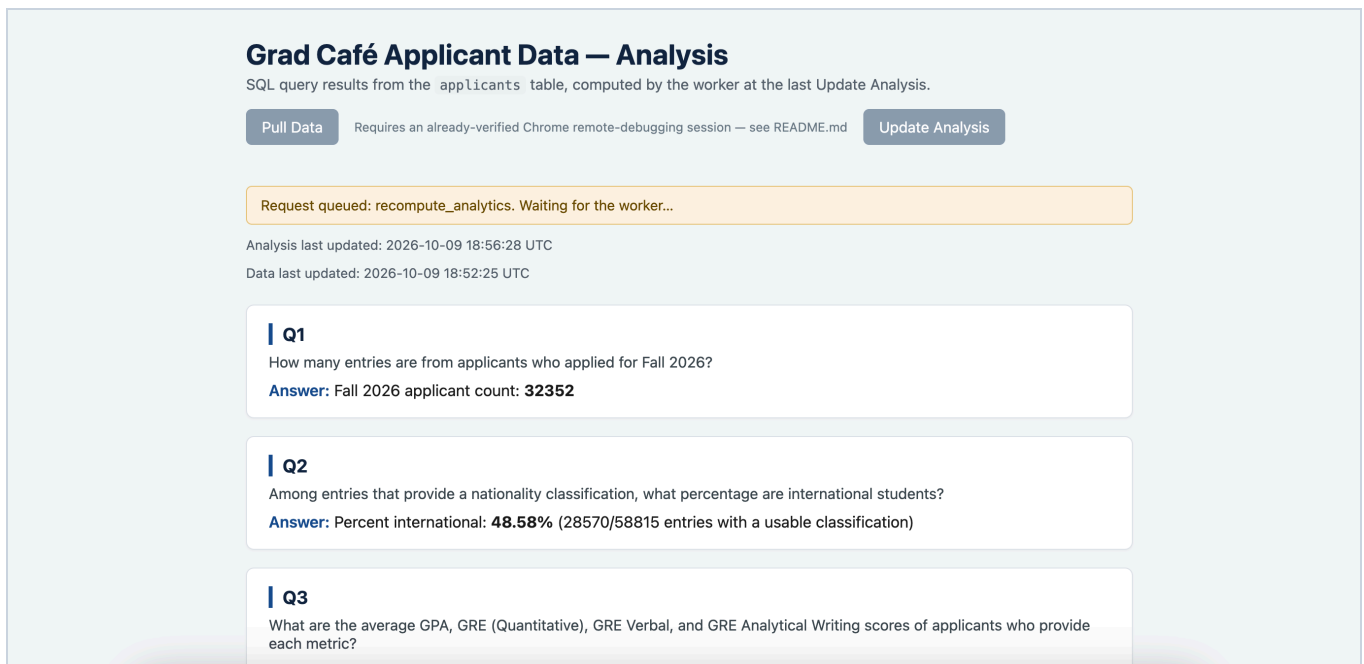


Figure 4. After pressing Update Analysis: the "Request queued: `recompute_analytics`" banner while the page waits for the worker.

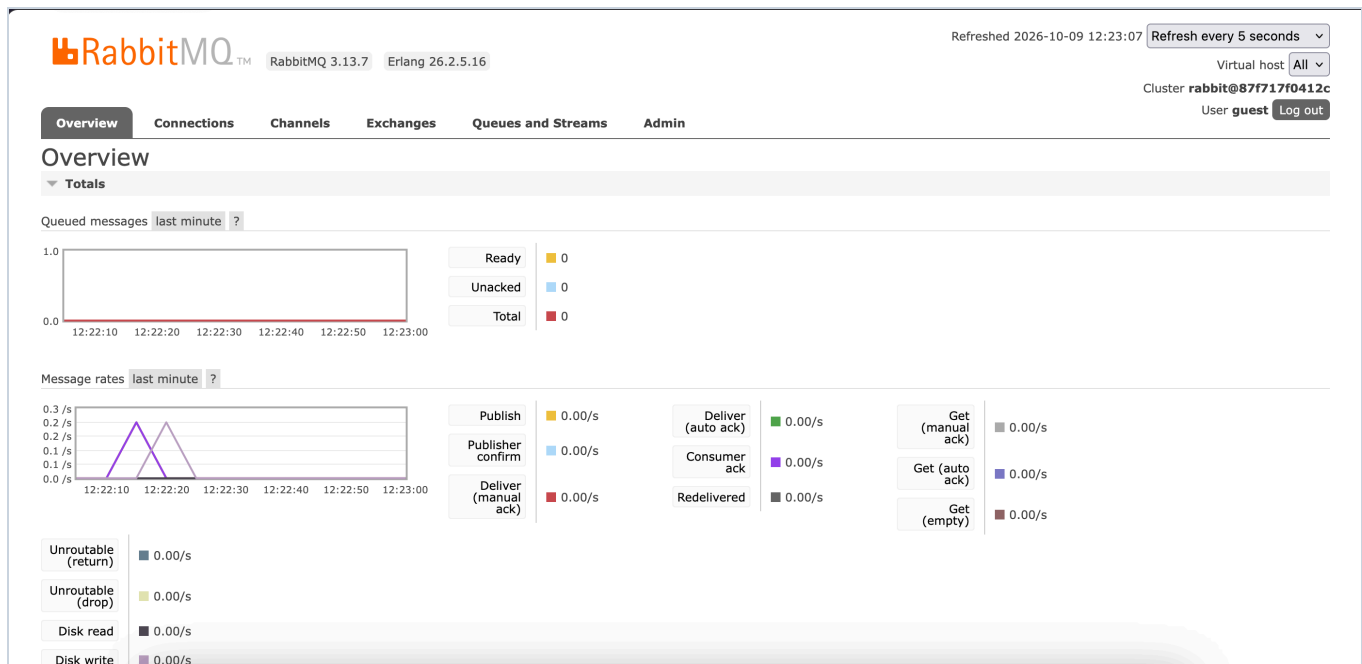


Figure 5. RabbitMQ management overview (RabbitMQ 3.13.7): no messages waiting.

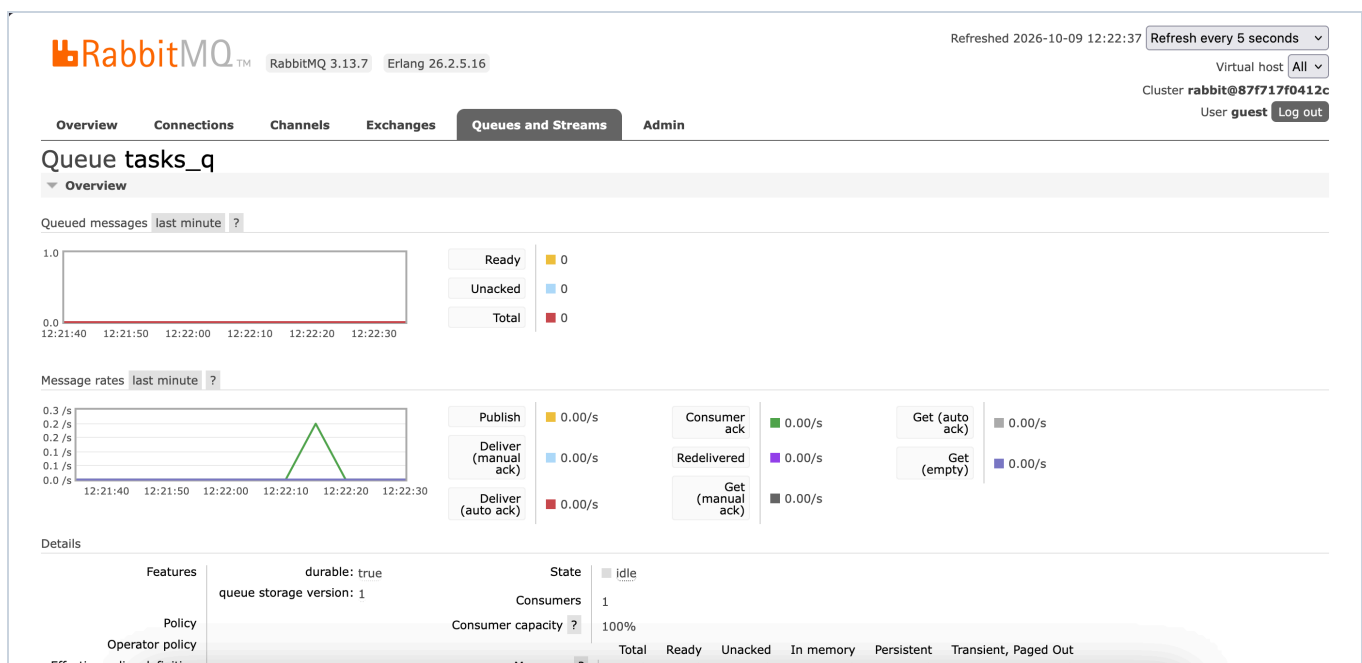


Figure 6. Queue `tasks_q`: durable, one consumer (the worker), nothing ready or unacknowledged.

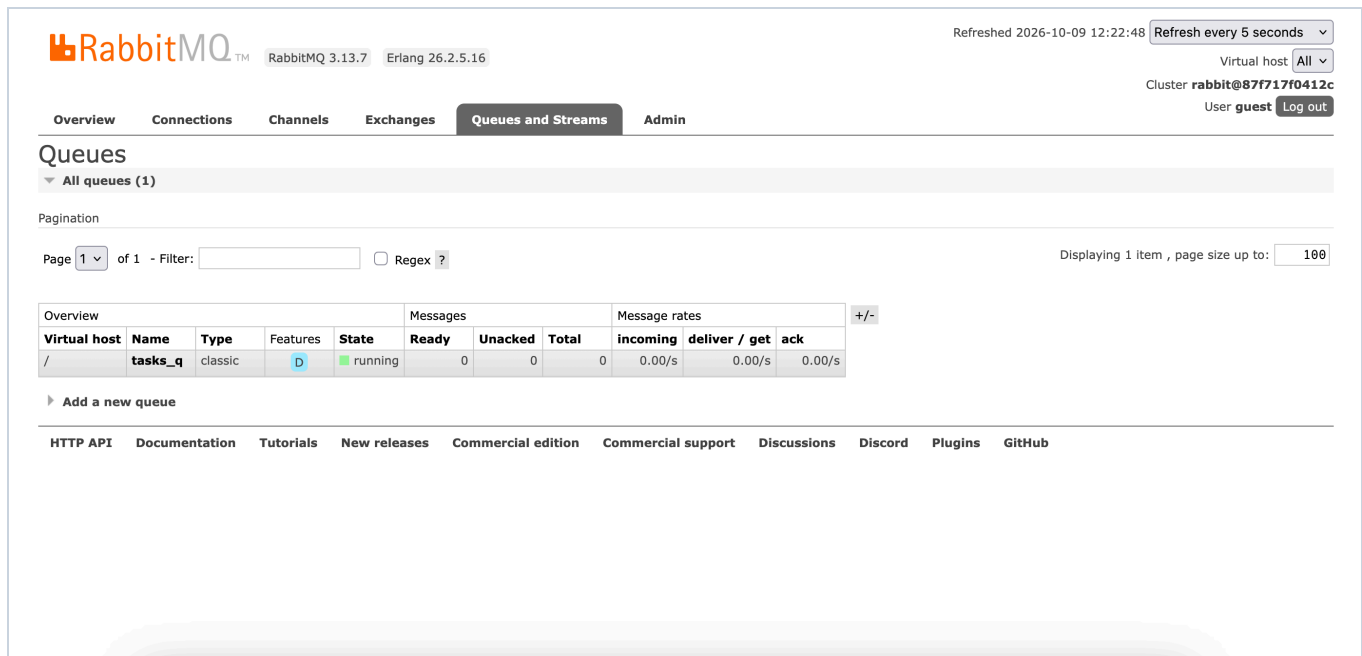


Figure 7. Queues list: `tasks_q`, classic, durable (D), running.

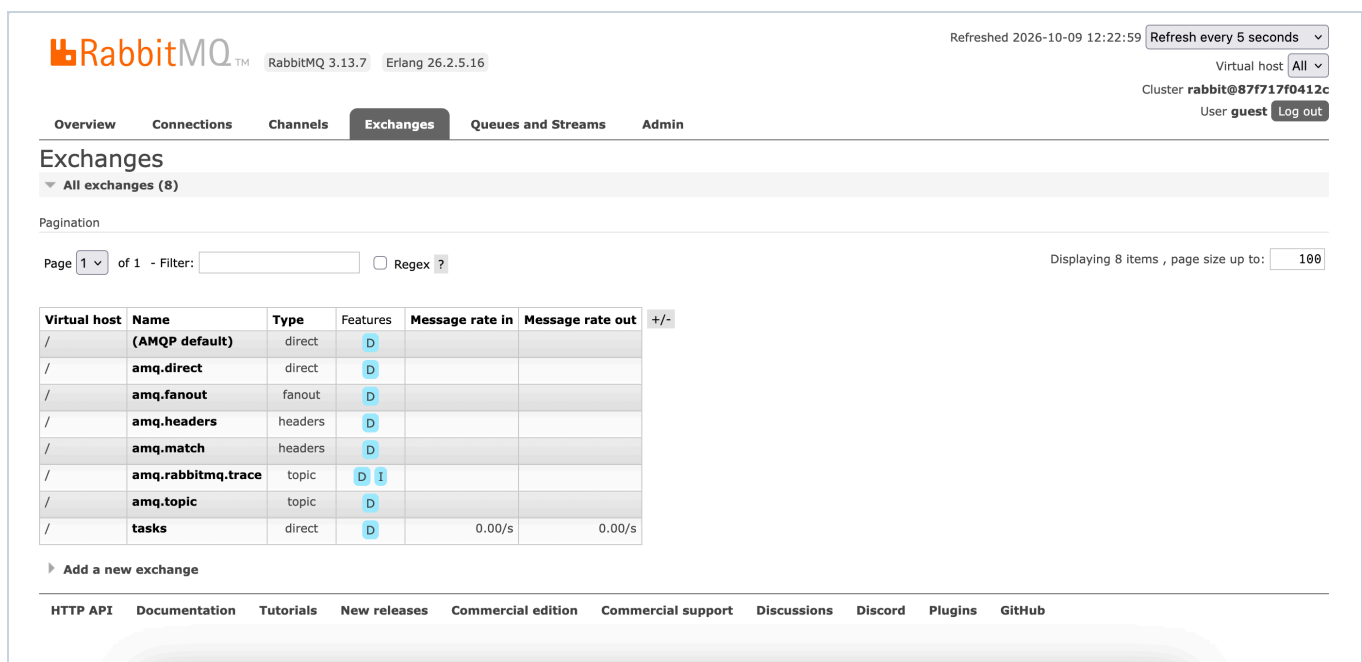


Figure 8. Exchanges: `tasks`, direct and durable, alongside RabbitMQ's built-in exchanges.

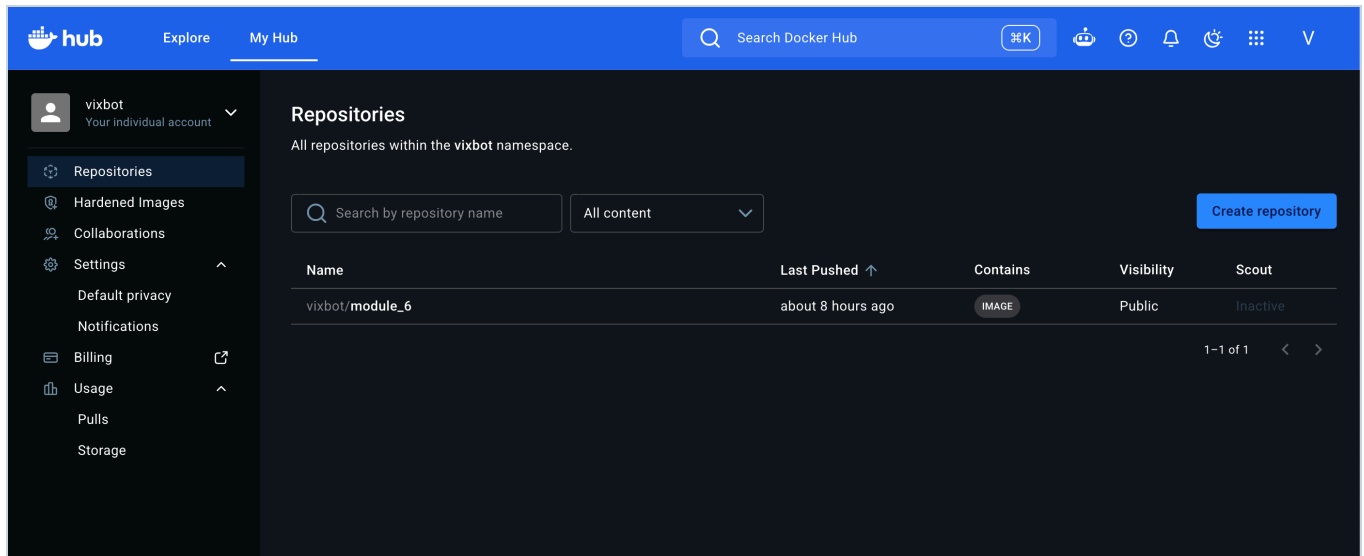


Figure 9. Docker Hub repository `vixbot/module_6`, visibility Public.

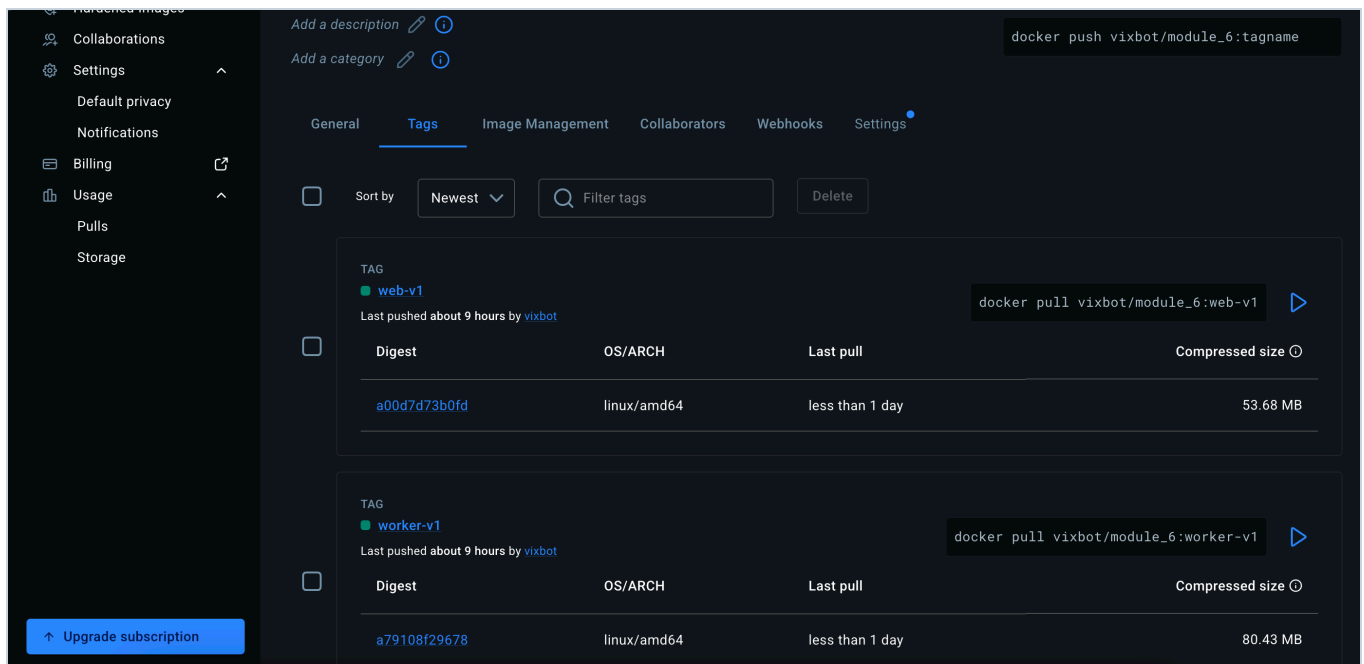


Figure 10. Tags `web-v1` and `worker-v1` (linux/amd64) with their pull commands.

Vix-T / jhu_software_concepts

Type to search

Code

Issues

Pull requests

Actions

Projects

Security and quality

Insights

Settings

Actions

New workflow

All workflows

Module 4 tests

Module 5 CI (dependency graph, Snyk)

Module 6 CI

Management

Caches

Attestations

Runners

Usage metrics

Performance metrics

All workflows

Showing runs from all workflows

Filter workflow runs

36 workflow runs

WorkflowEventStatusBranchActor

CI: Module 6 pylint and pytest workflow on Python 3.11; M...

Module 4 tests #25: Commit 9fd9238 pushed by Vix-T

main

3 minutes ago

53s

CI: Module 6 pylint and pytest workflow on Python 3.11; M...

Module 5 CI (dependency graph, Snyk) #10: Commit 9fd9238 pushed by Vix-T

main

3 minutes ago

46s

CI: Module 6 pylint and pytest workflow on Python 3.11; M...

Module 6 CI #1: Commit 9fd9238 pushed by Vix-T

main

3 minutes ago

41s

Module 6: chromedriver in worker image, scrape via host ...

Module 4 tests #24: Commit 43df7e0 pushed by Vix-T

main

29 minutes ago

41s

Module 6: chromedriver in worker image, scrape via host ...

Module 5 CI (dependency graph, Snyk) #9: Commit 43df7e0 pushed by Vix-T

main

29 minutes ago

47s

Module 6 Docker stage: per-service images, compose sta...

main

Today at 11:25 AM

Figure 11. GitHub Actions: Module 6 CI #1 green on commit 9fd9238, alongside the Module 5 (dependency graph, Snyk) and Module 4 workflows.